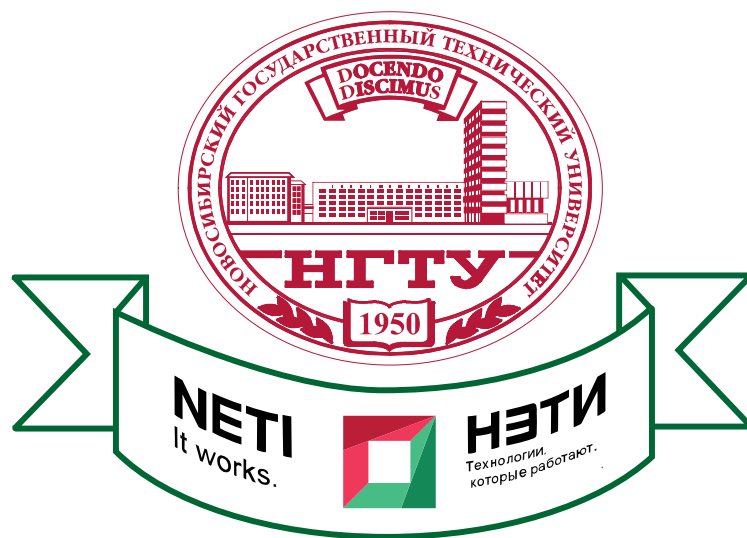
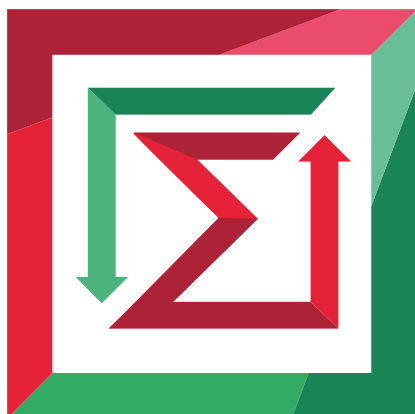


Министерство науки и высшего образования
Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



Кафедра теоретической и прикладной информатики
Лабораторная работа №2
По дисциплине «Программные средства защиты информации»

Блочные составные шифры. Сеть Фейстеля



Факультет:	ПМИ
Группа:	ПМИМ-51
Студенты:	Гавриленко Андрей, Яшков Иван
Вариант:	1
Преподаватель:	Авдеенко Татьяна Владимировна

Новосибирск, 2026

1. Задание

I. Реализовать приложение для шифрования, позволяющее выполнять следующие действия:

1. Шифровать данные при помощи сети Фейстеля:

- 1) шифруемый текст должен храниться в одном файле, а ключ шифрования – в другом;
- 2) приложение должно позволять выбирать способ получения подключей из заданного ключа шифрования:

а) для i -го раунда подключом V_i является цепочка из 32 подряд идущих бит заданного ключа, которая начинается с бита номер i , продолжается до последнего бита ключа и при его достижении циклически повторяется, начиная с 1 бита;

б) для i -го раунда, начиная с бита номер i , берётся цепочка из 8 подряд идущих бит ключа, которая является начальным значением для скремблера вида 0000 0011; подключом V_i является сгенерированная этим скремблером последовательность из 32 бит;

3) приложение должно позволять выбирать вид образующей функции:

а) функция – единичная $F(V_i) = V_i$;

б) функция имеет вид $F(V_i, X) = S(X) + V_i$, где $S(X)$ – левая часть шифруемого блока, на которую посредством операции XOR была наложена 32-битная последовательность, сгенерированная 16 разрядным скремблером вида 0100 0000 0000 0011;

4) зашифрованный текст должен сохраняться в файл;

5) в процессе шифрования предусмотреть возможность просмотра и изменения ключа, шифруемого и зашифрованного текстов в шестнадцатеричном и символьном виде.

2. Исследовать лавинный эффект (исследования проводить на одном блоке текста):

1) для бита, который будет изменяться, приложение должно позволять задавать его позицию (номер) в открытом тексте или в ключе;

2) приложение должно уметь после каждого раунда шифрования подсчитывать число бит, изменившихся в зашифрованном тексте при изменении одного бита в открытом тексте либо в ключе;

3) приложение может строить графики зависимости числа бит, изменившихся в зашифрованном тексте, от раунда шифрования, либо графики можно строить в стороннем ПО, но тогда приложение для шифрования должно сохранять в файл необходимую для построения графиков информацию.

II. Реализовать приложение для дешифрования, позволяющее выполнять следующие действия:

1. Дешифровать данные при помощи сети Фейстеля:

1) зашифрованный текст должен храниться в одном файле, ключ – в другом;

2) приложение должно позволять выбирать способ получения подключей из заданного ключа шифрования;

3) приложение должно позволять выбирать вид образующей функции;

4) расшифрованный текст должен сохраняться в файл;

5) в процессе дешифрования предусмотреть возможность просмотра и изменения ключа, зашифрованного и расшифрованного текстов в шестнадцатеричном и символьном виде.

III. С помощью реализованных приложений выполнить следующие задания:

1. Протестировать правильность работы разработанных приложений.

2. Исследовать лавинный эффект при изменении одного бита в открытом тексте и в ключе:

построить графики зависимостей числа бит, изменившихся в зашифрованном сообщении, от раунда шифрования при всех возможных комбинациях способов выбора ключа и образующей функции (всего должно быть построено 8 графиков).

3. Сделать выводы о проделанной работе.

2. Разработанное приложение

При выполнении лабораторной работы было написано консольное приложение на языке C, реализующее все функциональные требования, приведенные в задании под пунктами I, II. Некоторые функции программы приведены в конце отчета.

При запуске создаются три файла — message.txt для сообщения, key.txt для ключа, cipher.txt для шифротекста.

Основа приложения – сеть Фейстеля из 16 раундов с размером блока 64 бит.

Главное меню:

Choose a mode:

- (1) Encrypt data from 'message.txt' with key 'key.txt' and put new message in 'cipher.txt';
- (2) Decrypt data from 'cipher.txt' with key 'key.txt' and put new message in 'message.txt';
- (3) Examine the results of the Feistel network;
- (0) Exit.

>

1) При нажатии на (1), пользователь попадает в меню шифрования:

Choose a mode (current key method: 1, current func: 1):

- (1) Encrypt with Feistel;
 - (show <message.txt/cipher.txt/key.txt> <2/16/symbol>) Show a file in a specific form;
 - (write <message.txt/cipher.txt/key.txt> <2/16/symbol> <text>) Write text into a file in a specific form;
 - (pick_key <1/2>) Pick a subkey generation method (1 = cyclic 32 bits from key, 2 = 8 bits -> LFSR -> 32 bits);
 - (pick_func <1/2>) Pick a round function (1 = $F(V)=V$, 2 = $F(R,V) = S(R) \text{ XOR } V$);
 - (0) Back.

>

Здесь пользователь может задать сообщение, ключ и даже сообщение шифра с помощью символьного, 2-ичного или 16-ичного ввода, посмотреть эти значения в символьном, 2-ичном или 16-ичном виде, а также выбрать способ получения подключей и вид образующей функции.

2) При нажатии на (2), выполняется дешифрование текста в cipher.txt.

Здесь пользователь может задать сообщение, и сообщение шифра с помощью символьного, 2-ичного или 16-ичного ввода, задать ключ (начальное значение скремблера) в 2-ичном, 10-ичном и 16-ичном виде, посмотреть эти значения в соответствующих видах, выбрать один из двух скремблеров, а также выполнить шифрование, при котором результат будет положен в файл cipher.txt. При шифровании генерируется ключ из выходных битов скремблера в количестве битов исходного сообщения.

2) При нажатии на 3, пользователь попадает в меню исследования лавинного эффекта:

Choose a mode:

- (1 <bit_pos[0..63]>) Run avalanche-effect research for the given bit position (8 CSVs);
- (0) Back.

>

При нажатии <1 позиция_бита> создается 8 CSV файлов для просмотра графиков зависимости количества измененных битов от номера раунда.

3. Задания из пункта 3:

1. Протестировать правильность работы разработанных приложений.

Текст и ключ были записаны в соответствующие файлы:

> write message.txt symbol hello, world!!!! andrey

> write key.txt 16 af1c3d67bb09cf

Результат шифрования (подключи: 1, функция: 1): d0v00G%*0w00G-y0/000E-

Результат шифрования (подключи: 1, функция: 2): 0 0#0[0L0Z0 30 00,

Результат шифрования (подключи: 2, функция: 1): 0\000J#0Q00íJ#0#0H#

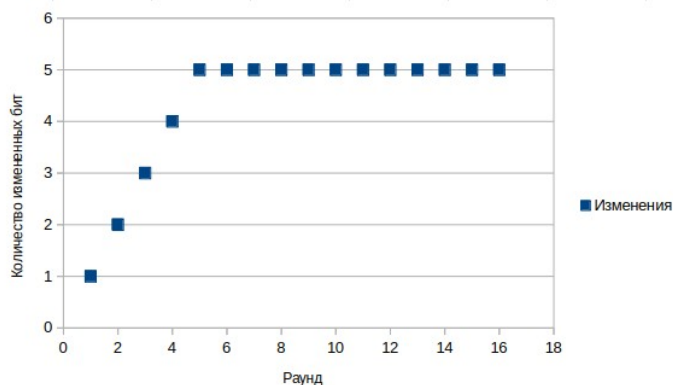
Результат шифрования (подключи: 2, функция: 2): 0e0u0N0000+0C0j(0 0#00

Дешифрование приводит к одному результату: hello, world!!!! andrey

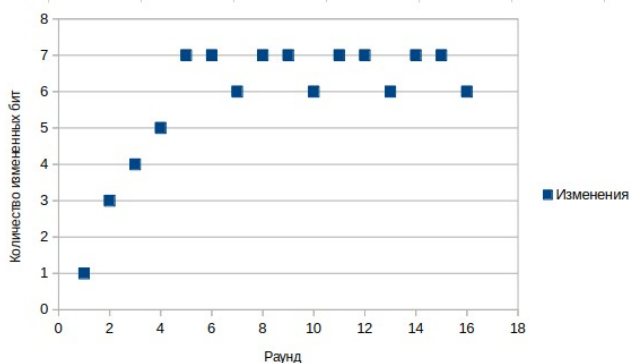
2. Исследовать лавинный эффект при изменении одного бита в открытом тексте и в ключе: построить графики зависимостей числа бит, изменившихся в зашифрованном сообщении, от раунда шифрования при всех возможных комбинациях способов выбора ключа и образующей функции (всего должно быть построено 8 графиков).

Далее приводятся результаты тестирования при изменении 5-го бита в открытом тексте и в ключе.

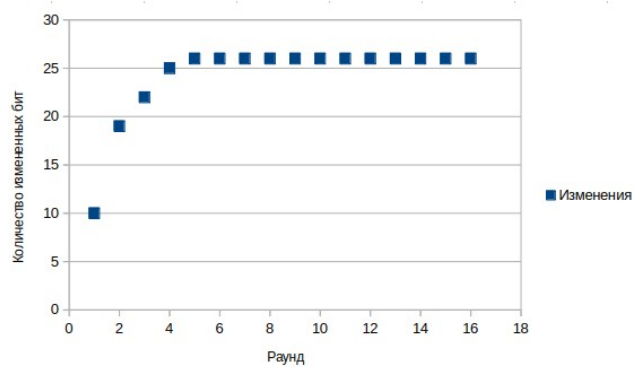
Изменение **в ключе**, подключа вычисляются по (а), вид функции (а):



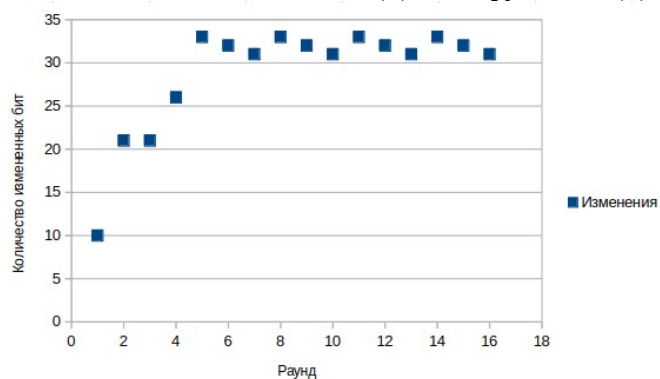
Изменение **в ключе**, подключа вычисляются по (а), вид функции (б):



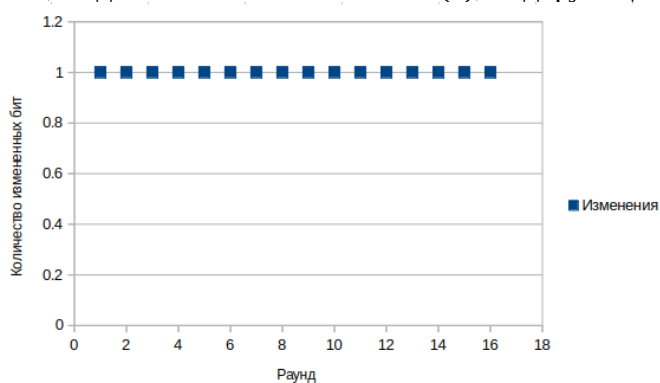
Изменение **в ключе**, подключа вычисляются по (б), вид функции (а):



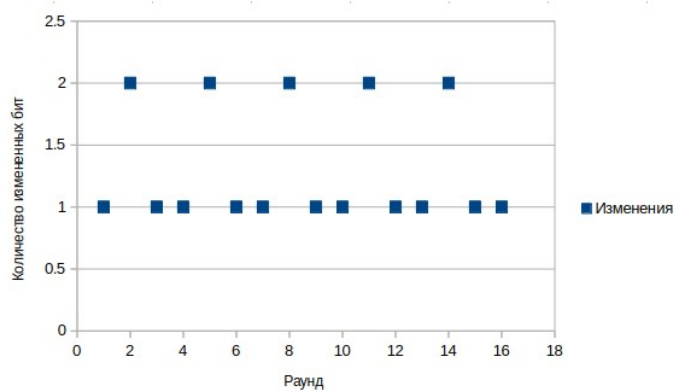
Изменение **в ключе**, подключа вычисляются по (б), вид функции (б):



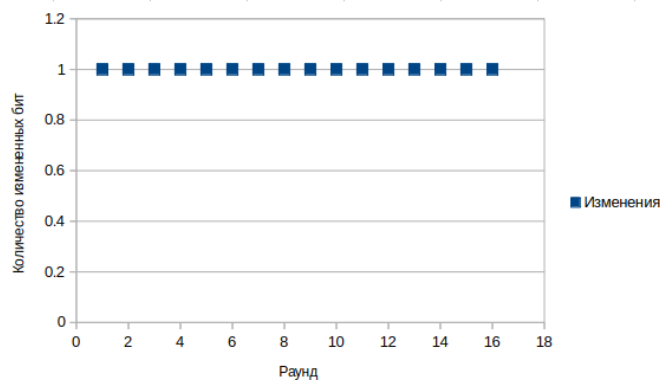
Изменение **в сообщении**, подключа вычисляются по (а), вид функции (а):



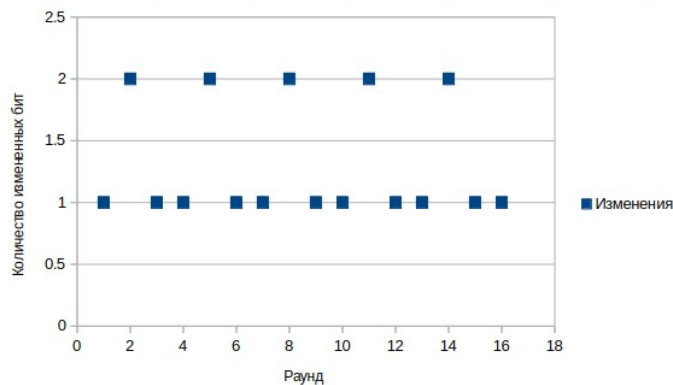
Изменение **в сообщении**, подключа вычисляются по (а), вид функции (б):



Изменение **в сообщении**, подключа вычисляются по (б), вид функции (а):



Изменение в сообщении, подключа вычисляются по (б), вид функции (б):



3. Сделать выводы о проделанной работе.

В данной работе было разработано приложение для шифрования/дешифрования сообщения с помощью сети Фейстеля. Глядя на графики исследования лавинного эффекта, можно сделать следующие выводы:

- 1) При изменении бита открытого текста чаще всего изменяется ровно 1 бит, так как образующие функции линейны;
- 2) При изменении бита ключа меняется много бит, так как каждый бит ключа влияет на создание нескольких подключей;
- 3) Наилучшей комбинацией оказалось шифрование с вычислением подключей и видом образующей функции с использованием скремблера. Данная комбинация, при изменении всего одного бита ключа, дает вплоть до 32 изменений в битах за 16 раундов, что является половиной блока.
- 4) При выборе способа вычисления подключей №2, при изменении n -го бита в ключе, изменения в сообщении начинаются только с раунда $n - 8$, так как влияние измененного бита начинается только с этого раунда. На первом раунде для подключа берутся 1-8 биты ключа, на втором – 2-9 и т. д., из-за этого какой-нибудь, например, 11-й измененный бит начнет влиять только на 4-ом раунде (биты 4-11 основного ключа).

4. Несколько основных функций

1) Функция подсчета одного раунда сети Фейстеля:

```
static void feistel_round(unsigned char* L, unsigned char* R,
                        const unsigned char* K,
                        const feistel_settings* settings) {
    unsigned char f_out[FEISTEL_HALF_BYTES];
    if (settings->func_type == 1)
        round_func_a(R, K, f_out);
    else
        round_func_b(R, K, f_out);
}
```

```

unsigned char new_R[FEISTEL_HALF_BYTES];
xor4(new_R, L, f_out);

memcpy(L, R, FEISTEL_HALF_BYTES);
memcpy(R, new_R, FEISTEL_HALF_BYTES);
}

```

где

FEISTEL_HALF_BYTES = 4 байта = 32 бита = полблока

```

typedef struct {
    int key_method;
    int func_type;
} feistel_settings;

```

round_func_a и round_func_b – соответствующие отображающие функции:

```

static void round_func_a(const unsigned char* R, const unsigned char* K,
                        unsigned char out[FEISTEL_HALF_BYTES]) {
    (void)R;
    memcpy(out, K, FEISTEL_HALF_BYTES);
}

static void round_func_b(const unsigned char* R, const unsigned char* K,
                        unsigned char out[FEISTEL_HALF_BYTES]) {
    unsigned short state = LFSR16_INIT;
    unsigned char seq[FEISTEL_HALF_BYTES];
    memset(seq, 0, FEISTEL_HALF_BYTES);
    for (int i = 0; i < 32; i++) {
        int bit = lfsr16_next(&state);
        set_bit_in_bytes(seq, i, bit);
    }
    for (int i = 0; i < FEISTEL_HALF_BYTES; i++) out[i] = R[i] ^ seq[i] ^ K[i];
}

```

2) Функция полного прохода для одного блока:

```

static void process_block(
    const unsigned char* in, unsigned char* out, const unsigned char* key,
    const feistel_settings* settings, int decrypt,
    unsigned char round_outputs[FEISTEL_ROUNDS][FEISTEL_BLOCK_BYTES]) {
    unsigned char subkeys[FEISTEL_ROUNDS][FEISTEL_HALF_BYTES];
    generate_subkeys(key, settings, subkeys);

    unsigned char L[FEISTEL_HALF_BYTES], R[FEISTEL_HALF_BYTES];
    memcpy(L, in, FEISTEL_HALF_BYTES);
    memcpy(R, in + FEISTEL_HALF_BYTES, FEISTEL_HALF_BYTES);

    for (int r = 0; r < FEISTEL_ROUNDS; r++) {
        int idx = decrypt ? (FEISTEL_ROUNDS - 1 - r) : r;
        feistel_round(L, R, subkeys[idx], settings);

        if (round_outputs) {
            memcpy(round_outputs[r], L, FEISTEL_HALF_BYTES);
            memcpy(round_outputs[r] + FEISTEL_HALF_BYTES, R, FEISTEL_HALF_BYTES);
        }
    }

    memcpy(out, R, FEISTEL_HALF_BYTES);
    memcpy(out + FEISTEL_HALF_BYTES, L, FEISTEL_HALF_BYTES);
}

```

где:

FEISTEL_ROUNDS = 16 – количество раундов

FEISTEL_BLOCK_BYTES = 8 байт = 64 бит – количество бит в одном блоке

а функция generate_subkeys (и дополнительные к ней функции get_subkey_a, gen_subkey_b) – функция вычисления подключей в соответствии с выбранным режимом:

```
void generate_subkeys(
    const unsigned char* key, const feistel_settings* settings,
    unsigned char subkeys[FEISTEL_ROUNDS][FEISTEL_HALF_BYTES]) {
    memset(subkeys, 0, FEISTEL_ROUNDS * FEISTEL_HALF_BYTES);
    for (int r = 1; r <= FEISTEL_ROUNDS; r++) {
        if (settings->key_method == 1)
            gen_subkey_a(key, r, subkeys[r - 1]);
        else
            gen_subkey_b(key, r, subkeys[r - 1]);
    }
}

static void gen_subkey_a(const unsigned char* key, int round,
    unsigned char out[FEISTEL_HALF_BYTES]) {
    memset(out, 0, FEISTEL_HALF_BYTES);
    int start_bit = round - 1;
    int key_bits = FEISTEL_KEY_BYTES * 8;
    for (int i = 0; i < 32; i++) {
        int bit = get_key_bit(key, (start_bit + i) % key_bits);
        set_bit_in_bytes(out, i, bit);
    }
}

static void gen_subkey_b(const unsigned char* key, int round,
    unsigned char out[FEISTEL_HALF_BYTES]) {
    memset(out, 0, FEISTEL_HALF_BYTES);
    int start_bit = round - 1;
    int key_bits = FEISTEL_KEY_BYTES * 8;

    unsigned char seed = 0;
    for (int i = 0; i < 8; i++) {
        int bit = get_key_bit(key, (start_bit + i) % key_bits);
        seed |= (unsigned char)(bit << (7 - i));
    }

    for (int i = 0; i < 32; i++) {
        int bit = lfsr8_next(&seed);
        set_bit_in_bytes(out, i, bit);
    }
}
```

3) Функция для сохранения результатов исследования лавинного эффекта:

```
static void write_csv(const char* fname, const unsigned char* rounds_orig,
    const unsigned char* rounds_mod) {
    int fd = open(fname, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open csv");
        return;
    }
    char line[64];
    int len = snprintf(line, sizeof(line), "round,bits_changed\n");
    write(fd, line, len);

    for (int r = 0; r < FEISTEL_ROUNDS; r++) {
        const unsigned char* a = rounds_orig + r * FEISTEL_BLOCK_BYTES;
        const unsigned char* b = rounds_mod + r * FEISTEL_BLOCK_BYTES;
        int d = count_diff_bits(a, b, FEISTEL_BLOCK_BYTES);
        len = snprintf(line, sizeof(line), "%d,%d\n", r + 1, d);
        write(fd, line, len);
    }
    close(fd);
}
```